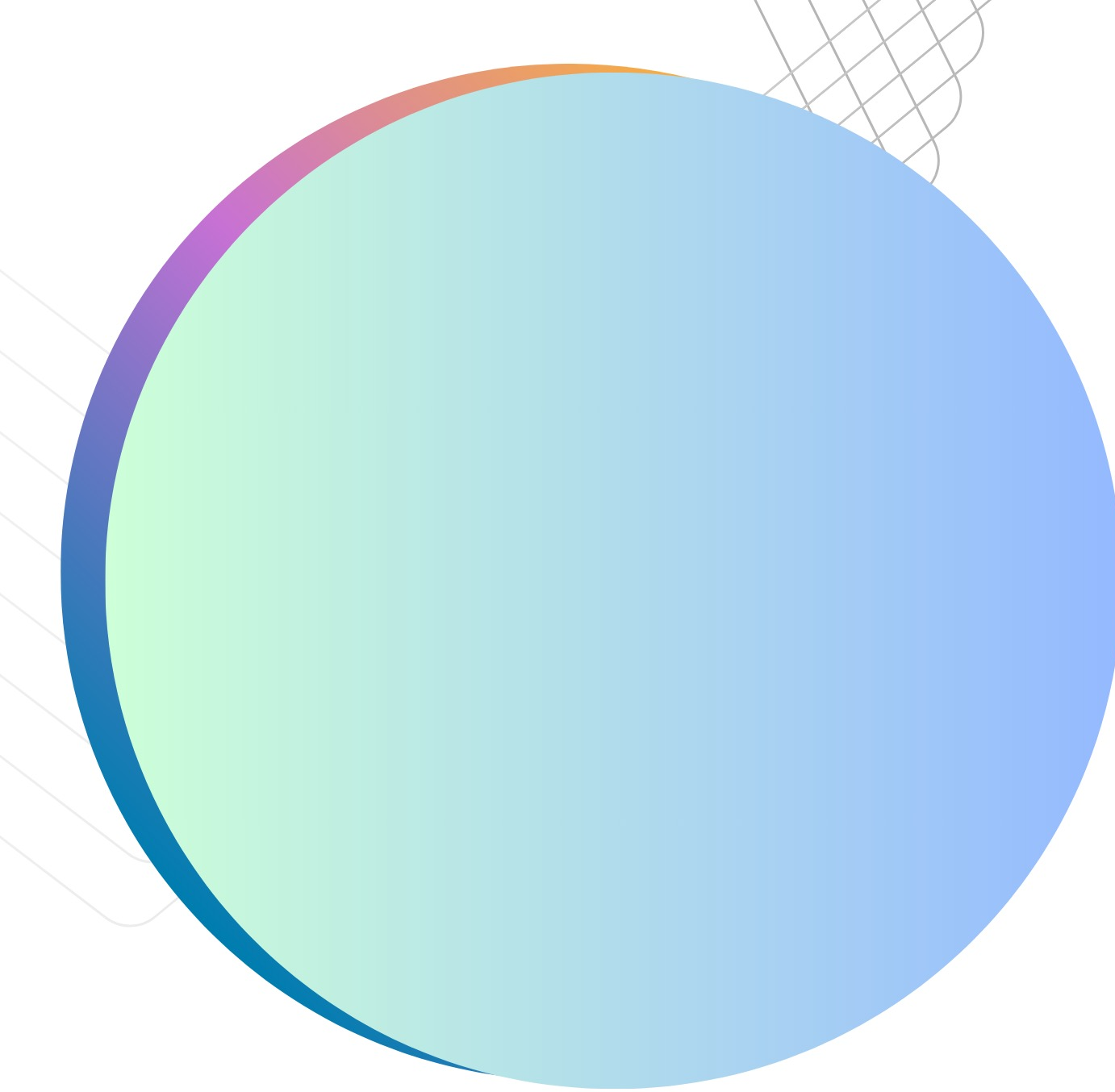




Bridge Pattern

Abstraction? Implementation? Sound scary?



About me

- Software Engineer at WiX
 - Co-org Codeus
- 



Bridge is a structural design pattern that lets you split a large class or a set of closely related classes into two separate hierarchies—abstraction and implementation—which can be developed independently of each other.



Problem



Payment Types

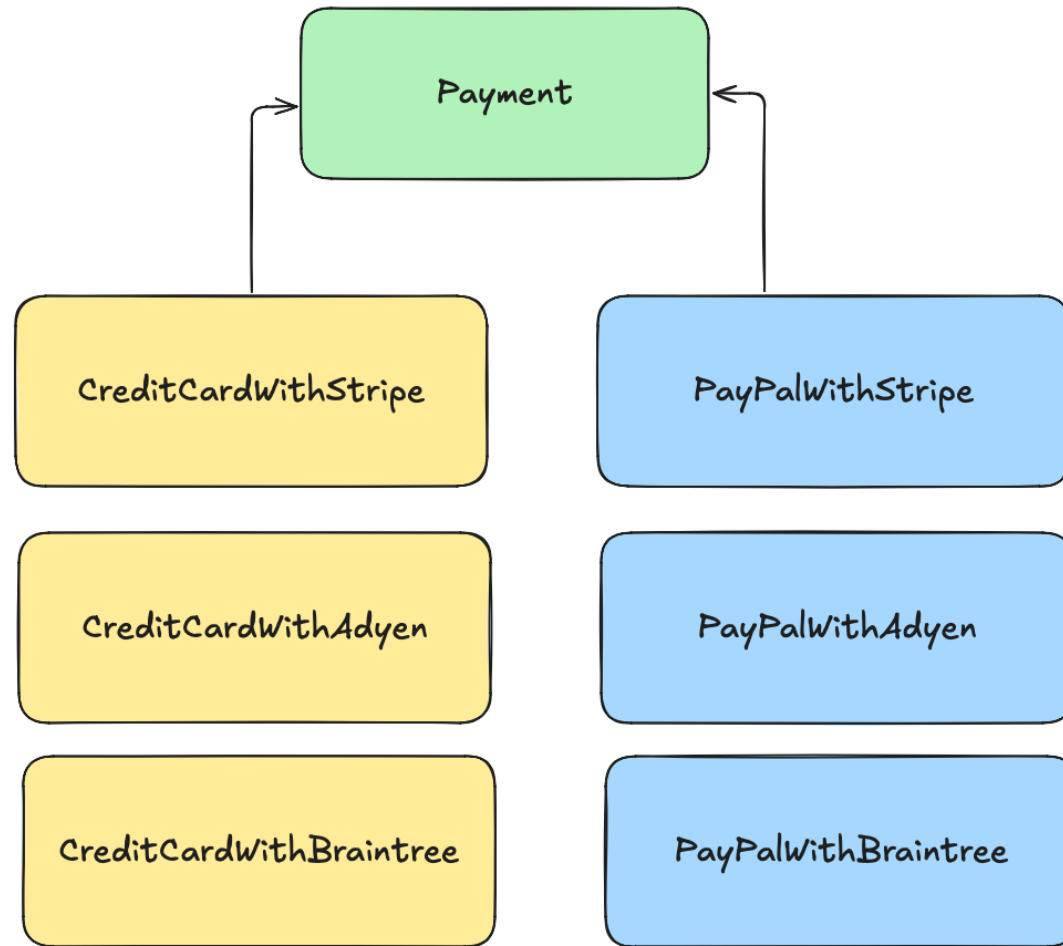


Payment Gateways

stripe

adyen

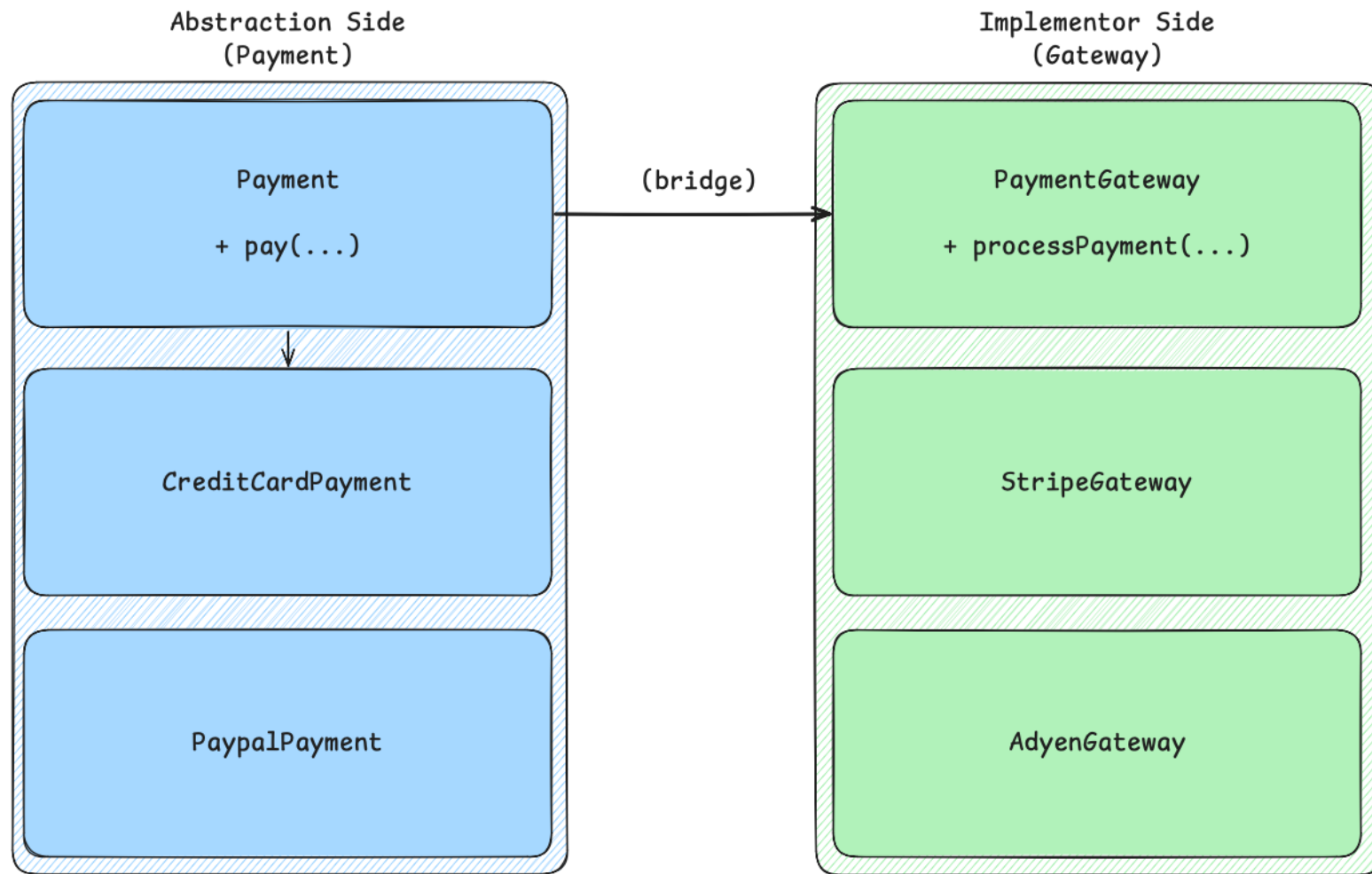
Braintree



2 (payment types) * 3 (payment gateway) = 6 classes

Solution





Implementor

```
interface PaymentGateway {  
    void processPayment(double amount);  
}
```

Concrete Implementors

```
class StripeGateway implements PaymentGateway {  
    @Override  
    public void processPayment(double amount) {  
        System.out.println("Processing $" + amount + " via Stripe.");  
    }  
}
```

```
class AdyenGateway implements PaymentGateway {  
    @Override  
    public void processPayment(double amount) {  
        System.out.println("Processing $" + amount + " via Adyen.");  
    }  
}
```

Abstraction

```
abstract class Payment {  
    protected PaymentGateway gateway;  
  
    public Payment(PaymentGateway gateway) {  
        this.gateway = gateway;  
    }  
  
    public abstract void pay(double amount);  
}
```

Implementations

```
class CreditCardPayment extends Payment {  
    public CreditCardPayment(PaymentGateway gateway) {  
        super(gateway);  
    }  
}
```

@Override

```
public void pay(double amount) {  
    System.out.println("Paying with Credit Card...");  
    gateway.processPayment(amount);  
}  
}
```

```
class PayPalPayment extends Payment {  
    public PayPalPayment(PaymentGateway gateway) {  
        super(gateway);  
    }  
}
```

@Override

```
public void pay(double amount) {  
    System.out.println("Paying with PayPal...");  
    gateway.processPayment(amount);  
}  
}
```

Pros and Cons



You can create platform-independent classes and apps.



The client code works with high-level abstractions. It isn't exposed to the platform details.



Open/Closed Principle.



Single Responsibility Principle.



You might make the code more complicated by applying the pattern to a highly cohesive class.



Thank you



Author: Dmytro Halchenko

